

Siddhant Kumar Upmanyu

Software Engineer (2022–2024) → Senior Software Engineer (2024–2026) → Senior Systems Architect (Apr 2026–Present)

Location: Bengaluru, India • sku20.dev • GitHub • LinkedIn • siddhantkumarupmanyu@gmail.com • Zenodo Preprint

PLATFORM ARCHITECTURE & TECHNICAL SCOPE

- **Platform Architecture & Technical Ownership:** Direct the end-to-end technical strategy and engineering roadmap for the platform, reporting to the Head of Product. Autonomously drive system architecture: identifying foundational risks, conducting R&D spikes, making core infrastructure decisions, and executing zero-downtime database and platform migrations on live production traffic.
- **Infrastructure & Platform Governance:** Hold administrative and architectural responsibility across the operational footprint: corporate domain and DNS governance (GoDaddy), Google Cloud Organization Administration (IAM policies, VPC topologies, and project lifecycles), on-prem staging bare metal, and production compute clusters.
- **Cloud FinOps & Infrastructure Economics:** Directly responsible for infrastructure capacity planning and cloud spend. Analyzed historical utilization to structure a 3-year Google Cloud Committed Use Discount (CUD) that slashed compute costs; currently driving next-gen hardware migrations (legacy N1 to modern N4 instances) to maximize throughput per cloud dollar.
- **Cross-Team Technical Standards:** Define and maintain platform data contracts, binary IoT transport protocols, kernel-level traffic routing, and architectural boundaries—authoring the technical specifications that firmware, hardware, mobile, and backend teams implement against.

2026 Promoted to Senior Systems Architect | Declarative Cloud, Kernel Networking & Toolmaker

In April 2026, I was promoted to Senior Systems Architect, reporting directly to the Head of Product. My mandate covers platform architecture, cloud infrastructure, kernel networking, and long-term technical strategy across IoT firmware, mobile interfaces, and distributed backends.

Cloud Organization Administration & Compute Lifecycle (N1 → N4)

- **Administrative & Platform Governance:** Manage foundational platform resources as Google Cloud Organization Administrator (IAM policies, project topologies, VPC networks, firewall rules) and administrator for corporate DNS zone files and domains via GoDaddy.
- **Compute Lifecycle & Next-Gen Hardware Migration (N1 → N4):** As the 2023 3-year CUD agreement approached completion, I led the technical evaluation and capacity planning to migrate core production workloads from legacy Google Cloud N1 instances to next-generation N4 machine types—benchmarking instructions-per-cycle (IPC) gains, memory throughput, and optimizing cost-per-workload.

Feature Flags Engine & Controlled Rollouts (2026)

- **The Problem:** Releasing high-impact changes across cloud APIs and mobile/firmware devices carried high blast radii if unexpected edge cases emerged in the field.
- **The Solution:** Engineered an internal, lightweight feature flags system directly into the backend—enabling dynamic runtime evaluation, percentage-based rollouts, and instant kill-switches, allowing teams to dark-launch new capabilities without backend redeployments.

100% Declarative Infrastructure for Marketing Website (Non-Core): Terraform / OpenTofu (Cloud Run & Cloud SQL)

- **The Scope & Isolation:** Non-core public marketing website. The company's public marketing website was previously hosted on manually configured virtual machines, lacking clean isolation from core platform resources and declarative reproducibility.
 - **The Solution:** Decoupled the marketing website completely from core backend infrastructure into its own isolated Google Cloud project driven 100% by Terraform / OpenTofu IaC: executed a zero-downtime DNS and workload cutover, migrating the marketing site onto containerized Google Cloud Run paired with managed Google Cloud SQL (PostgreSQL), codifying VPC networking, service accounts, IAM bindings, secrets, and database provisioning with zero manual cloud console mutations—ensuring marketing workloads have zero blast radius on core IoT backend systems.
-

Bare-Metal Homelab & Cloud-Native Engineering (Proxmox, K8s, Nomad)

- **Preparing for Platform Containerization:** Built an on-prem bare-metal testbed running Proxmox VE to chart the company's future containerization roadmap: automated VM provisioning via `cloud-init`, bootstrapped and benchmarked Kubernetes clusters from scratch (evaluating CNI, CSI, and Pod Identities), and benchmarked HashiCorp Nomad vs. Kubernetes (EKS).
-

Concurrency Overhaul: Dynamic IoT-Aware ThreadPool vs. ForkJoinPool

- **The Bottleneck:** Default JVM concurrency models struggled with bursty IoT packet waves, either wasting memory on idle threads or inducing packet drops during sudden surges.
 - **The Implementation:** Replaced the legacy pool with a custom, load-adaptive dynamic ThreadPool that dynamically scales worker capacity in response to real-time incoming packet velocity and queue saturation. Eliminated legacy technical debt: obsolete dynamic TCP port provisioning flows and redundant in-memory state tracking.
-

Low-Level Kernel Networking & Edge Protocol Translation

- **In-Kernel NAT64 & Dual-Stack Routing (Jool + nftables):** Addressed a critical network partition where cellular IoT clients operated exclusively over IPv6 while backend systems ran on IPv4. Ruled out userspace proxies (Nginx/HAProxy) that would drain burstable CPU credits; deployed the **Jool** Linux kernel module for stateful NAT64 (RFC 7915) via Explicit Address Mapping Tables (EAMT) alongside `nftables` DNAT, cleanly partitioning non-overlapping source port ranges on a shared public IPv4.
 - **Kernel-Level Packet Reflection & Dynamic Rate-Limiting:** Offloaded UDP NAT traversal echo responses directly into the Linux kernel using `nftables` packet reflection at prerouting priority (not `rack`). Implemented kernel-level DDoS/abuse protection using `nftables` dynamic sets (`limit rate over 10/second burst 20 packets at prerouting priority -301`) to drop traffic sweeps before socket allocation.
 - **Academic Preprint (Zenodo):** Authored and published *Connection-Agnostic Presence Tracking for Stateless Distributed Backends*: formulated a Redis-based architecture using sorted sets scored by expiration deadlines and throttled batch writes to track IoT device online/offline transitions across stateless backends with mathematically bounded detection latency.
-

Zero-Downtime Storage Architecture: Decoupling In-DB Payloads to Google Cloud Storage (GCS)

- **The Architecture Challenge:** Legacy systems stored raw binary image payloads (Base64) directly inside primary MongoDB collections, degrading transactional throughput, ballooning nightly backup archives, and polluting WiredTiger cache memory.
- **Multi-Phase Rollout & Dual-Write Strategy:** Conceived and orchestrated a zero-downtime, multi-stage migration lifecycle coordinating mobile clients and backend services: designed dual-write API contracts with document-level guardrail flags (`imageMigrated`) preventing legacy client overwrites, engineered an asynchronous backfill pipeline streaming decoded historical blobs to GCS with automatic stream MIME sniffing and immutable caching headers (`Cache-Control: public, max-age=31536000, immutable`), and handled corrupted legacy payloads with clean asset fallbacks.
- **Graceful Cutover & Database Purge:** Maintained backward compatibility during a 2–3 day mobile client rollout buffer (~99% adoption), followed by a second deployment deprecating legacy Base64 endpoints and running MongoDB `$unset` operations to completely purge in-document blobs and migration flags.
- **The Measurable Outcome:** Zero user downtime across live production. Slashed primary database storage by **54x** (3.6 GB → 67 MB, a 98% reduction in backup footprint), dropped runtime server memory footprint by **89%** (4.6 GB → ~500 MB), and eliminated binary blob overhead across all collections.

Production Database Administration (MongoDB): Schema Refactoring & Compound Index Optimization

- **The Challenge (Sustained Heavy-Load Production):** As IoT write velocity and active homes surged, the primary production MongoDB cluster faced elevated disk I/O and memory pressure in the WiredTiger cache caused by legacy schema anti-patterns and accumulated index sprawl.
- **Query Plan Audits & Index Hygiene:** Profiled live production query traffic using `explain` execution stats and slow query logs. Audited and pruned redundant, low-selectivity, and duplicate indexes that were imposing heavy write overhead on high-frequency IoT inserts and wasting working-set RAM.
- **Compound Key Engineering & Schema Optimization:** Engineered selective compound indexes applying strict key ordering (Equality, Sort, Range — ESR pattern), ensuring high-frequency queries were satisfied entirely within index trees and eliminating expensive collection scans and in-memory sorts. Refactored legacy document schemas to curb unbounded growth and eliminate fragmented on-disk document allocations.

2025 Senior Software Engineer | Production Observability, Modern Infra & High-Performance Pipelines

In 2025, the research from late 2024 materialized into production infrastructure. I moved the company away from single-node disk-dependent operations toward dedicated observability, declarative infrastructure, and purpose-built ingestion pipelines.

Production Observability Rollout: Grafana & Loki (Retiring `hlogger`)

- **The Architectural Philosophy:** Clarified the fundamental distinction between *"Is the service up?"* (external blackbox probes) vs. *"Why is the service struggling?"* (internal contextual metrics).
- **The Implementation:** Provisioned a dedicated, isolated observability VM. Deployed and tuned Grafana, Loki, and Mimir fed by Alloy (running Blackbox HTTP/TCP probes alongside database and application metrics exporters). Managed all configurations in a git-tracked directory (pragmatic GitOps/IaC). Gracefully retired `hlogger` and disk log dumping.

High-Volume Telemetry Migration: ClickHouse, Delta+ZSTD Codecs & GCS Offloading

- **The Inode Crisis:** The production VM was facing recurring disk exhaustion from telemetry saved across deeply nested two-letter directory trees (/aa/bb/cc/. . .), causing catastrophic filesystem inode bloat.
 - **Zero-Downtime Migration & Partition Backfilling:** Engineered a zero-downtime live migration pipeline: rather than running slow, memory-intensive INSERT INTO queries, backfilled and restored historical data at the partition level while live device writes continued uninterrupted. Devised full operational management with partition-optimized backup and disaster recovery restore strategies (archiving and restoring yearly partitions to/from GCS).
 - **Schema & Codec Engineering:** Modeled columnar ClickHouse schemas (device_health, pal_2024 activity logs, and crm_logs) with tight data types: FixedString(23/29), LowCardinality(String), Delta encoding, and ZSTD compression. Slashed storage footprint by over 95% (compressing device_health telemetry from 17 GB down to 300 MB on disk) with composite primary keys optimizing block skip scans.
-

Ending "Testing in Production": On-Prem Staging Environment & Parity

- **The Dangerous Status Quo:** Historically, changes were deployed directly against live production systems, causing high deployment churn and regressions.
 - **The Architecture:** Configured a dedicated on-prem server behind a static public IP replicating production topology. Established a strict deployment promotion gate where all features and refactors were verified on staging before receiving production tickets—dramatically reducing hotfixes.
-

Decoupled Data Architecture: Go + gRPC Device Health Service (Encapsulating ClickHouse)

- **The Architectural Decision:** Refused to tightly couple the main Kotlin/Spring Boot monolith with ClickHouse drivers. Built a standalone, lightweight Go microservice communicating over binary gRPC with strict Protocol Buffers contracts, deployed on the auxiliary VM.
-

SNode Architecture & The Git-Based Knowledge Base Revolution

- **Virtual Node Composite Abstraction:** Designed the architectural federation allowing disjoint physical hardware devices (e.g. multiple dimmers) to aggregate into a single composite Virtual Node (a subtype of SNode) that behaves as one unified device.
 - **Establishing the Knowledge Base:** Spearheaded the company's first Git-based Knowledge Base: a version-controlled repository of technical specifications, protocol definitions, and API contracts that firmware and app engineers reviewed and built against before shipping hardware.
-

Multi-Tier Rate Limiting & High-Ingestion Telemetry R&D

- **Defense in Depth (Nginx + Bucket4j):** Configured reverse-proxy rate limiting in Nginx (HTTP 429) and token-bucket application rate limiting via Bucket4j inside the gRPC Activity Log service.
 - **Tackling Cardinality (VictoriaMetrics & VictoriaLogs):** Benchmarked VictoriaMetrics and VictoriaLogs on staging to evaluate their cardinality-agnostic architecture over Loki/Mimir when handling thousands of dynamic IoT device IDs.
-

Cloud-to-Cloud Integration: Yale Smart Locks (2025)

- Integrated Yale smart locks natively into the cloud platform: designed cloud-to-cloud OAuth integration, handled device state synchronization, access filtering, and real-time push notifications for remote lock and unlock events.
-

Technical Recruitment & Engineering Standards (Late 2025 – 2026)

- Partnered with HR to establish the company's first structured technical Job Description and candidate evaluation rubric for backend systems. Designed practical coding assessments evaluating systems thinking and TDD, conducted engineering interviews, and successfully hired an engineer into the team.

By 2024, my ownership spanned from daily database forensics and bug fixes to core infrastructure reliability. I transitioned from surviving production fires to proactively eliminating security vulnerabilities, profiling JVM runtime internals, and hardening automated recovery mechanisms.

Zero-Trust Internal Tooling: `hlogger` (Go, mTLS & Custom PKI)

- **The Security Flaw:** Firmware engineers previously had raw SSH access to production Linux instances to read device logs—a severe audit and security vulnerability.
- **The Solution:** Built a secure client-server diagnostic tool in Go using Mutual TLS (mTLS): engineered a complete PKI hierarchy from scratch (Root CA, Intermediate CA, client certificates with CRL support). The server enforced mutual authentication, audited commands, and streamed filtered logs without exposing interactive shells.

Production Resilience & The "Hospital Bed" Recovery Incident

- **The Vulnerability:** While hospitalized in mid-2024, an engineer triggered a deployment swap during high lag, corrupting iptables routing rules and taking down production.
- **Triage & Remediation:** The CEO video-called directly from my hospital room, aiming a camera at the engineer's terminal; I walked them through flushing corrupted rules and restoring traffic. Upon recovery, I overhauled the swap tooling to be strictly idempotent, self-healing, and guarded with sanity checks.

JVM Deep Profiling & Garbage Collection Engineering

- **Profiling Runtime Internals:** Instrumented the runtime with `async-profiler`, Java Mission Control (JMC), and VisualVM, generating flame graphs under live traffic. Mitigated allocation churn with pooled buffers and `ThreadLocal` allocations; pinned heap boundaries (`-Xms = -Xmx` at 4GB) and tuned GC for low pause times.
- **Hot-Path Zero-Allocation Formatting:** Identified CPU hotspots in binary packet decoding loops driven by `String.format`; refactored to Java 17 `HexFormat` and bitwise operations, cutting per-packet CPU overhead.
- **Redis Client Forensics:** Evaluated Redis drivers (Jedis vs. Lettuce) for async I/O; debugged silent "ghost connections" and poisoned connection pool states in Redis Pub/Sub that caused cascading application timeouts.

The Observability & High-Ingestion Data Odyssey: Discovering ClickHouse

- Manually evaluated distributed logging and analytics engines (ELK vs. OpenSearch vs. Loki; Druid vs. Hadoop vs. ClickHouse). Identified ClickHouse as the optimal engine for high-throughput IoT time-series telemetry.
- Attended the inaugural Apache Kafka meetup in Bangalore (first official Kafka event in India) and Thoughtworks distributed systems events.

Device Provisioning Rewrite & Hardware Replacement Operations (Late 2024 – Early 2025)

- Extracted and overhauled provisioning logic from the monolith: implemented strict node validation assertions and completely rewrote the WiFi onboarding state machine. Built the replacement flow for failed hardware nodes, allowing hardware swaps without losing room mappings or automations.

Consumer-Driven Contract Testing Spike: Pact (2024–2025)

- Investigated Consumer-Driven Contract Testing (Pact) across 2024 and 2025 for OTA firmware delivery APIs, researching automated verification pipelines (`publish pact` → `can-i-deploy` → `release`) to eliminate integration regressions.

In January 2023, the founding engineer departed, leaving me as the sole custodian of the entire backend, the production virtual machines, and cloud infrastructure. With no safety net, I was immediately thrust into heavy SRE firefighting—tackling cascading outages, resource starvation, and kernel bottlenecks while keeping feature development alive.

Linux Systems Forensics: Surviving File Descriptor Depletion & I/O Starvation

- **The Crisis:** Recurring production failures where the backend was unable to allocate threads, locking out SSH sessions and forcing emergency VM reboots.
- **Diagnostics & Fix:** Used `sar`, `vmstat`, `iotop`, and `pidstat` to diagnose log flooding and `iowait` starvation. Discovered that shell `ulimit` and PAM were bypassed by `systemd`; reconfigured `systemd` unit limits (`LimitNOFILE=262144`), reloaded daemons, and tuned disk I/O scheduling using `nice` and `ionice`.

Database Connection Pool Forensics (MongoDB)

- Traced recurring database timeouts to an internal auxiliary service leaking unpooled MongoDB connections on requests; re-architected the service to use pooled connection lifecycles, immediately stabilizing database clusters.

Platform Expansion: Multi-Hub Automations & Ecosystem Integrations

- Extended the scene and rule automation engine to support multi-hub environments—handling nested scene fragments for large command payloads and multi-hub scene sync.
- Engineered and stabilized cloud-to-cloud voice integrations across Google Home and Amazon Alexa.

Critical Production State-Sync Remediation (Cloud & Hub Data Divergence)

- Discovered a severe, months-old bug in legacy sync code where flawed ID generation silently corrupted scene and rule automations on physical hubs. Rewrote the sync logic for deterministic conflict resolution and executed zero-downtime live production migration and state-reconciliation scripts across thousands of active homes without breaking customer automations or dropping hub connectivity.

Cloud FinOps & Infrastructure Economics: 3-Year Committed Use Discount (CUD)

- Reported cloud spend forecasts directly to the CEO. Analyzed baseline vs. burst utilization and committed to a 3-year Google Cloud Committed Use Discount (CUD) on core compute instances—locking in aggressive cost reductions for the company over a multi-year horizon.

Delivery Infrastructure: Bitbucket to GitHub Actions Migration

- Ported all build, test, and release pipelines from Bitbucket to GitHub Actions while preserving automated webhook zero-downtime cutovers without service interruptions.

Architecture Research & Extreme Programming (XP) Foundations

- Researched horizontal scalability patterns (Kafka vs. lightweight queues, Kubernetes trade-offs) and immersed in Extreme Programming (XP) and London-style TDD as a non-negotiable safety net.

I joined as Software Engineer and sole dedicated backend engineer. The only other backend contributor was a founding engineer splitting focus across Android and backend systems. There was no backend team to lean on, no deployment pipeline, and years of legacy infrastructure. I stepped into immediate, end-to-end ownership of backend architecture, delivery pipelines, and production reliability.

Zero-Downtime Blue-Green Deploys & Automated Delivery

- **The Reality on the Ground:** Deployments were completely manual (SFTP'ing Jetty WAR files and restarting processes), causing recurring downtime.
- **Automated Delivery & Kernel Switching:** Engineered an automated CI/CD pipeline on Bitbucket with automated server webhooks. Cut over traffic at the Linux kernel level using `iptables` DNAT in both PREROUTING (external) and OUTPUT (loopback) chains, explicitly flushing the conntrack state table for immediate cut-over without dropping active requests.

IoT Hub Load Simulation & The C10K Awakening

- Tasked with simulating hundreds of concurrent IoT hubs replaying traffic logs. Encountered the C10K bottleneck where 1:1 thread-per-socket allocation exhausted JVM stacks; dove into Java NIO (`Selector`, `SocketChannel`), Go goroutines, and early Project Loom preview builds—fundamentally reshaping my mental model toward async and non-blocking I/O.

Spring 4 to Spring Boot 2.7.5 Migration: Unlocking TDD

- Untangled legacy XML and bean configurations to leap directly from legacy Spring 4 (JDK 8) to Spring Boot 2.7.5 (JDK 17) in the pre-AI era, cutting the modernized monolith over live via the kernel blue-green pipeline with zero downtime. The primary driver was developer velocity: unlocking modern Spring test slices and making Test-Driven Development (TDD) viable across the backend.

Apple HomeKit Integration Spike: Protocol Forensics & HAP Bridging

- Forked and adapted an open-source Java implementation of Apple's HomeKit Accessory Protocol (HAP). Deep-dived into mDNS/Bonjour discovery, cryptographic pairing exchanges (SRP and Curve25519), and session encryption.

Open-Source Systems & Research Projects

stopgap Kotlin · Maven Central (~2.8.0)

Modern microservice framework built on **Helidon SE (Nima)** + **Project Loom** virtual threads. Features compile-time dependency injection via **KSP** (eliminating runtime reflection overhead) and an integrated three-tier testing harness (unit → in-process integration server → Docker E2E via Testcontainers).

assertgo Go · Generics

Type-safe testing assertion library built with modern Go generics. Provides a fluent API, chainable negation (`Not()`), custom matchers, and zero external dependencies.

relay Zig · Systems

Low-level TCP server implemented in pure Zig with a test-driven approach. Explores raw POSIX socket descriptors, manual memory management without libc runtime dependencies, port binding (SO_REUSEADDR), and preventing broken-pipe crashes (SIGPIPE suppression via MSG_NOSIGNAL).

hrh Rust · CLI ('cargo install')

Helm Release Helper — engineered during Kubernetes research to enable lean, declarative Helm releases without operator bloat. Reads YAML declarations and executes `helm upgrade --install` with diff previews and atomic rollback guarantees.

avoid Shell / Linux · OS

Minimal, purpose-built Linux distribution based on Void Linux for server recovery and lean headless appliances. Builds and publishes bootable `.img.gz` and `.qcow2` images via automated GitHub Actions pipelines.

c_oop C · TDD Spike

Object-Oriented Programming and London-style TDD in pure C (written entirely pre-AI). Implements struct polymorphism via function-pointer interface tables, heap-allocated lifecycle constructors, and isolated unit test harnesses.

Technical Thought Leadership & Systems Writing

Authored 20+ in-depth technical post-mortems and distributed systems essays published at **sku20.dev/blog**:

Kernel Networking & Edge Routing

Betting on NAT64 Over a Proxy • Negotiating with Jool • Low-Level UDP Echo Server for NAT Traversal via nftables • Zero-Downtime Deployments with iptables

Concurrency & Runtime Internals

The 100ms Password • Java Exceptions Swallowed: The ThreadPool Trap • ThreadLocal Optimizations and Project Loom
• Optimizing Hex Formatting: String.format to Java 17 HexFormat

Distributed Systems & Database Reliability

Decoupling Blobs: A Zero-Downtime Migration from MongoDB to GCS • The Ghost Connection: Poisoned Redis Pub/Sub Connection Pools • Rate-Limiting: Flow Control vs. Quota Control • Hunting for a UDP Load Balancer

Technical Competencies & Education

Languages:	Kotlin, Java, Go, Rust, Zig, C, Shell, SQL
Frameworks & Protocols:	Spring Boot, Helidon SE (Nima), Project Loom, gRPC, Protocol Buffers, REST
Cloud & Infrastructure:	Linux (Debian, Ubuntu), Google Cloud (Cloud Run, Cloud SQL, GCS, VPC), AWS (EKS), Terraform, Docker, Proxmox VE, GitHub Actions, Bitbucket
Databases & Storage:	ClickHouse, MongoDB, PostgreSQL, Redis
Observability & SRE:	Grafana, Loki, Mimir, Prometheus, VictoriaMetrics, VictoriaLogs, Alloy, async-profiler, JMC, VisualVM, Flame Graphs, sar, vmstat, iotop
Networking & Security:	Socket programming (TCP/UDP), iptables, nftables, NAT64 (Jool), NAT traversal, mTLS & PKI (Root/Intermediate CA, CRL)
Methodology:	Extreme Programming (XP), London-Style TDD, Consumer-Driven Contracts (Pact), Testcontainers

Education: Bachelor of Computer Applications (BCA) • 2019 – 2022